ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

1 Introduction

Current approaches to object recognition make essential use of machine learning methods. To improve their performance, we can collect larger datasets, learn more powerful models, and use better techniques for preventing overfitting. Until recently, datasets of labeled images were relatively small — on the order of tens of thousands of images (e.g., NORB [16], Caltech-101/256 [8, 9], and CIFAR-10/100 [12]). Simple recognition tasks can be solved quite well with datasets of this size, especially if they are augmented with label-preserving transformations. For example, the current-best error rate on the MNIST digit-recognition task (<0.3%) approaches human performance [4]. But objects in realistic settings exhibit considerable variability, so to learn to recognize them it is necessary to use much larger training sets. And indeed, the shortcomings of small image datasets have been widely recognized (e.g., Pinto et al. [21]), but it has only recently become possible to collect labeled datasets with millions of images. The new larger datasets include LabelMe [23], which consists of hundreds of thousands of fully-segmented images, and ImageNet [6], which consists of over 15 million labeled high-resolution images in over 22,000 categories.

To learn about thousands of objects from millions of images, we need a model with a large learning capacity. However, the immense complexity of the object recognition task means that this problem cannot be specified even by a dataset as large as ImageNet, so our model should also have lots of prior knowledge to compensate for all the data we don't have. Convolutional neural networks (CNNs) constitute one such class of models [16, 11, 13, 18, 15, 22, 26]. Their capacity can be controlled by varying their depth and breadth, and they also make strong and mostly correct assumptions about the nature of images (namely, stationarity of statistics and locality of pixel dependencies). Thus, compared to standard feedforward neural networks with similarly-sized layers, CNNs have much fewer connections and parameters and so they are easier to train, while their theoretically-best performance is likely to be only slightly worse.

基于深度卷积神经网络的 ImageNet 图像分类

亚历克斯・克里热夫斯基
多伦多大学
kriz@cs.utoronto.ca

伊利亚・苏茨克维尔
多伦多大学
ilya@cs.utoronto.ca

杰弗里・E・欣顿
多伦多大学
hinton@cs.utoronto.ca

抽象的

我们训练了一个大型深度卷积神经网络，用于将 ImageNet LSVRC-2010 竞赛中的 120 万张高分辨率图像分类到 1000 个不同的类别中。在测试数据集上，我们实现了 37.5% 的 top-1 错误率和 17.0% 的 top-5 错误率，这显著优于之前的最佳结果。该神经网络拥有 6000 万个参数和 65 万个神经元，由五个卷积层（其中一些卷积层后接最大池化层）和三个全连接层组成，最终连接一个 1000 路 softmax 层。为了加快训练速度，我们使用了非饱和神经元和高效的 GPU 卷积运算实现。为了减少全连接层的过拟合，我们采用了一种名为“dropout”的最新正则化方法，该方法已被证明非常有效。我们还参加了 ILSVRC-2012 竞赛，并取得了前 5 名最佳测试错误率 15.3% 的佳绩，而第二名的测试错误率为 26.2%。

1 介绍

当前目标识别方法主要依赖于机器学习技术。为了提升性能，我们可以收集更大的数据集，学习更强大的模型，并采用更有效的过拟合预防技术。直到最近，带标签图像的数据集规模仍然相对较小，通常只有数万张图像（例如，NORB [16]、Caltech-101/256 [8, 9] 和 CIFAR-10/100 [12]）。对于简单的识别任务，这种规模的数据集已经能够很好地完成，尤其是在使用标签保持变换进行数据增强的情况下。例如，目前在 MNIST 数字识别任务上的最佳错误率（<0.3%）已经接近人类水平 [4]。然而，现实场景中的物体具有相当大的变异性，因此，为了学习识别它们，必须使用更大的训练集。事实上，小型图像数据集的不足之处早已被广泛认可（例如，Pinto 等人 [21]），但直到最近，收集包含数百万张图像的带标签数据集才成为可能。新的大型数据集包括 LabelMe [23]，其中包含数十万张完全分割的图像；以及 ImageNet [6]，其中包含超过 1500 万张标注的高分辨率图像，涵盖超过 22000 个类别。

为了从数百万张图像中学习数千个物体，我们需要一个具有强大学习能力的模型。然而，物体识别任务的复杂性极高，即使像 ImageNet 这样庞大的数据集也无法完全描述这个问题，因此我们的模型还需要大量的先验知识来弥补数据上的不足。卷积神经网络 (CNN) 就是这样一类模型 [16, 11, 13, 18, 15, 22, 26]。它们的学习能力可以通过改变深度和广度来控制，并且它们对图像的性质做出了强有力的、大多正确的假设（即统计平稳性和像素依赖性的局部性）。因此，与具有相似层数的标准前馈神经网络相比，CNN 的连接数和参数数要少得多，因此更容易训练，而它们的理论最佳性能可能只略逊一筹。

Despite the attractive qualities of CNNs, and despite the relative efficiency of their local architecture, they have still been prohibitively expensive to apply in large scale to high-resolution images. Luckily, current GPUs, paired with a highly-optimized implementation of 2D convolution, are powerful enough to facilitate the training of interestingly-large CNNs, and recent datasets such as ImageNet contain enough labeled examples to train such models without severe overfitting.

The specific contributions of this paper are as follows: we trained one of the largest convolutional neural networks to date on the subsets of ImageNet used in the ILSVRC-2010 and ILSVRC-2012 competitions [2] and achieved by far the best results ever reported on these datasets. We wrote a highly-optimized GPU implementation of 2D convolution and all the other operations inherent in training convolutional neural networks, which we make available publicly[1]. Our network contains a number of new and unusual features which improve its performance and reduce its training time, which are detailed in Section 3. The size of our network made overfitting a significant problem, even with 1.2 million labeled training examples, so we used several effective techniques for preventing overfitting, which are described in Section 4. Our final network contains five convolutional and three fully-connected layers, and this depth seems to be important: we found that removing any convolutional layer (each of which contains no more than 1% of the model's parameters) resulted in inferior performance.

In the end, the network's size is limited mainly by the amount of memory available on current GPUs and by the amount of training time that we are willing to tolerate. Our network takes between five and six days to train on two GTX 580 3GB GPUs. All of our experiments suggest that our results can be improved simply by waiting for faster GPUs and bigger datasets to become available.

2 The Dataset

ImageNet is a dataset of over 15 million labeled high-resolution images belonging to roughly 22,000 categories. The images were collected from the web and labeled by human labelers using Amazon's Mechanical Turk crowd-sourcing tool. Starting in 2010, as part of the Pascal Visual Object Challenge, an annual competition called the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) has been held. ILSVRC uses a subset of ImageNet with roughly 1000 images in each of 1000 categories. In all, there are roughly 1.2 million training images, 50,000 validation images, and 150,000 testing images.

ILSVRC-2010 is the only version of ILSVRC for which the test set labels are available, so this is the version on which we performed most of our experiments. Since we also entered our model in the ILSVRC-2012 competition, in Section 6 we report our results on this version of the dataset as well, for which test set labels are unavailable. On ImageNet, it is customary to report two error rates: top-1 and top-5, where the top-5 error rate is the fraction of test images for which the correct label is not among the five labels considered most probable by the model.

ImageNet consists of variable-resolution images, while our system requires a constant input dimensionality. Therefore, we down-sampled the images to a fixed resolution of 256×256. Given a rectangular image, we first rescaled the image such that the shorter side was of length 256, and then cropped out the central 256×256 patch from the resulting image. We did not pre-process the images in any other way, except for subtracting the mean activity over the training set from each pixel. So we trained our network on the (centered) raw RGB values of the pixels.

3 The Architecture

The architecture of our network is summarized in Figure 2. It contains eight learned layers — five convolutional and three fully-connected. Below, we describe some of the novel or unusual features of our network's architecture. Sections 3.1-3.4 are sorted according to our estimation of their importance, with the most important first.

[1] http://code.google.com/p/cuda-convnet/

尽管卷积神经网络（CNN）具有诸多吸引人的特性，且其局部架构效率相对较高，但将其大规模应用于高分辨率图像仍然成本过高。幸运的是，目前的 GPU，配合高度优化的二维卷积算法，已经足够强大，可以训练规模相当大的 CNN 模型。此外，诸如 ImageNet 等最新数据集包含足够多的标注样本，足以训练此类模型而不会出现严重的过拟合。

本文的具体贡献如下：我们在 ILSVRC-2010 和 ILSVRC-2012 竞赛 [2] 中使用的 ImageNet 子集上训练了迄今为止规模最大的卷积神经网络之一，并取得了迄今为止在这些数据集上报道的最佳结果。我们编写了一个高度优化的 GPU 实现，用于实现二维卷积以及训练卷积神经网络所需的所有其他操作，并将其公开提供。我们的网络包含许多新颖且独特的特征，这些特征可以提高其性能并缩短训练时间，详见第 3 节。即使使用 120 万个带标签的训练样本，我们网络的规模仍然导致过拟合问题十分突出，因此我们采用了几种有效的过拟合预防技术，详见第 4 节。我们最终的网络包含五个卷积层和三个全连接层，这种深度似乎至关重要：我们发现，移除任何一个卷积层（每个卷积层包含的参数不超过模型参数的 1%）都会导致性能下降。

最终，网络规模主要受限于当前 GPU 的可用内存以及我们愿意接受的训练时间。我们的网络在两块 GTX 580 3GB GPU 上训练需要五到六天。所有实验都表明，只需等待更快的 GPU 和更大的数据集可用，我们的结果就能得到提升。

2　数据集

ImageNet 是一个包含超过 1500 万张高分辨率图像的数据集，这些图像被分为大约 22000 个类别。这些图像收集自网络，并由人工标注员使用亚马逊 Mechanical Turk 众包工具进行标注。自 2010 年起，作为 Pascal 视觉对象挑战赛的一部分，每年都会举办 ImageNet 大规模视觉识别挑战赛 (ILSVRC)。ILSVRC 使用 ImageNet 的一个子集，该子集包含大约 1000 张图像，涵盖 1000 个类别。总共有大约 120 万张训练图像、5 万张验证图像和 15 万张测试图像。

ILSVRC-2010 是 ILSVRC 数据集中唯一提供测试集标签的版本，因此我们的大部分实验都是基于此版本进行的。由于我们也参加了 ILSVRC-2012 竞赛，因此在第 6 节中，我们也报告了基于此版本数据集（测试集标签不可用）的结果。在 ImageNet 数据集中，通常报告两个错误率：top-1 和 top-5。其中，top-5 错误率是指正确标签不在模型认为最有可能的五个标签之列的测试图像所占的比例。

ImageNet 数据集包含不同分辨率的图像，而我们的系统需要恒定的输入维度。因此，我们将图像下采样到固定分辨率 256 × 256。对于矩形图像，我们首先将其缩放，使短边长度为 256，然后从结果图像中裁剪出中心 256 × 256 的图像块。除了从每个像素值中减去训练集的平均活动值之外，我们没有对图像进行任何其他预处理。因此，我们使用像素的（居中）原始 RGB 值来训练我们的网络。

3　建筑

图 2 总结了我们网络的架构。它包含八个学习层——五个卷积层和三个全连接层。下面，我们将描述我们网络架构的一些新颖或独特的特征。第 3.1 节至 3.4 节的内容按重要性排序，最重要的放在最前面。

[1] http://code.google.com/p/cuda-convnet/

3.1 ReLU Nonlinearity

The standard way to model a neuron's output f as a function of its input x is with $f(x) = \tanh(x)$ or $f(x) = (1 + e^{-x})^{-1}$. In terms of training time with gradient descent, these saturating nonlinearities are much slower than the non-saturating nonlinearity $f(x) = \max(0, x)$. Following Nair and Hinton [20], we refer to neurons with this nonlinearity as Rectified Linear Units (ReLUs). Deep convolutional neural networks with ReLUs train several times faster than their equivalents with $\tanh$ units. This is demonstrated in Figure 1, which shows the number of iterations required to reach 25% training error on the CIFAR-10 dataset for a particular four-layer convolutional network. This plot shows that we would not have been able to experiment with such large neural networks for this work if we had used traditional saturating neuron models.

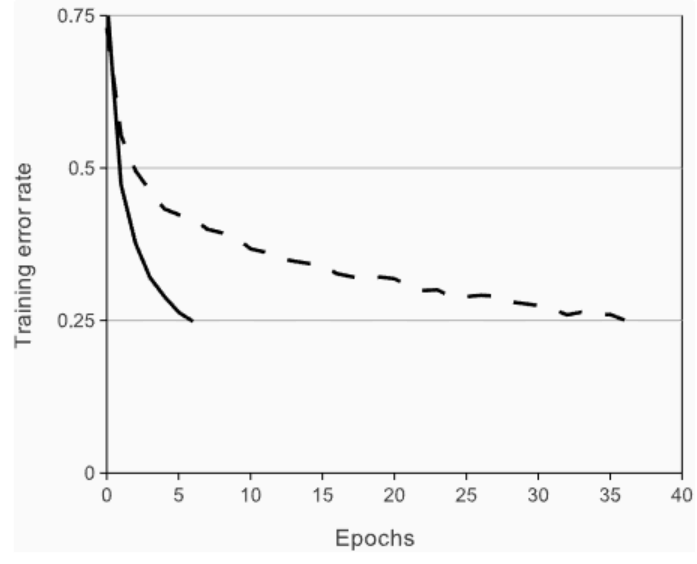


Figure 1: A four-layer convolutional neural network with ReLUs **(solid line)** reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with $\tanh$ neurons **(dashed line)**. The learning rates for each network were chosen independently to make training as fast as possible. No regularization of any kind was employed. The magnitude of the effect demonstrated here varies with network architecture, but networks with ReLUs consistently learn several times faster than equivalents with saturating neurons.

We are not the first to consider alternatives to traditional neuron models in CNNs. For example, Jarrett et al. [11] claim that the nonlinearity $f(x) = |\tanh(x)|$ works particularly well with their type of contrast normalization followed by local average pooling on the Caltech-101 dataset. However, on this dataset the primary concern is preventing overfitting, so the effect they are observing is different from the accelerated ability to fit the training set which we report when using ReLUs. Faster learning has a great influence on the performance of large models trained on large datasets.

3.2 Training on Multiple GPUs

A single GTX 580 GPU has only 3GB of memory, which limits the maximum size of the networks that can be trained on it. It turns out that 1.2 million training examples are enough to train networks which are too big to fit on one GPU. Therefore we spread the net across two GPUs. Current GPUs are particularly well-suited to cross-GPU parallelization, as they are able to read from and write to one another's memory directly, without going through host machine memory. The parallelization scheme that we employ essentially puts half of the kernels (or neurons) on each GPU, with one additional trick: the GPUs communicate only in certain layers. This means that, for example, the kernels of layer 3 take input from all kernel maps in layer 2. However, kernels in layer 4 take input only from those kernel maps in layer 3 which reside on the same GPU. Choosing the pattern of connectivity is a problem for cross-validation, but this allows us to precisely tune the amount of communication until it is an acceptable fraction of the amount of computation.

The resultant architecture is somewhat similar to that of the "columnar" CNN employed by Cireşan et al. [5], except that our columns are not independent (see Figure 2). This scheme reduces our top-1 and top-5 error rates by 1.7% and 1.2%, respectively, as compared with a net with half as many kernels in each convolutional layer trained on one GPU. The two-GPU net takes slightly less time to train than the one-GPU net[2].

[2]The one-GPU net actually has the same number of kernels as the two-GPU net in the final convolutional layer. This is because most of the net's parameters are in the first fully-connected layer, which takes the last convolutional layer as input. So to make the two nets have approximately the same number of parameters, we did not halve the size of the final convolutional layer (nor the fully-conneced layers which follow). Therefore this comparison is biased in favor of the one-GPU net, since it is bigger than "half the size" of the two-GPU net.

3.1 ReLU 非线性

将神经元的输出 f 建模为其输入 x 的函数的标准方法是使用 f(x) = tanh(x) 或 f(x) = (1 + e)。就梯度下降的训练时间而言，这些饱和非线性函数比非饱和非线性函数 f(x) = max(0, x) 慢得多。根据 Nair 和 Hinton [20] 的研究，我们将具有这种非线性函数的神经元称为修正线性单元 (ReLU)。使用 ReLU 的深度卷积神经网络的训练速度比使用 tanh 单元的等效网络快数倍。图 1 展示了这一点，图中显示了特定四层卷积神经网络在 CIFAR-10 数据集上达到 25% 训练误差所需的迭代次数。该图表明，如果我们使用传统的饱和神经元模型，我们将无法在本研究中使用如此大规模的神经网络进行实验。

我们并非第一个考虑在卷积神经网络（CNN）中使用传统神经元模型替代方案的研究者。例如，Jarrett 等人 [11] 声称，非线性函数 f(x) = |tanh(x)| 与他们提出的对比度归一化方法以及后续的局部平均池化在 Caltech-101 数据集上表现尤为出色。然而，在该数据集上，他们的主要关注点是防止过拟合，因此他们观察到的效果与我们在使用 ReLU 时所报告的加速训练集拟合能力有所不同。更快的学习速度对在大型数据集上训练的大型模型的性能有着显著的影响。

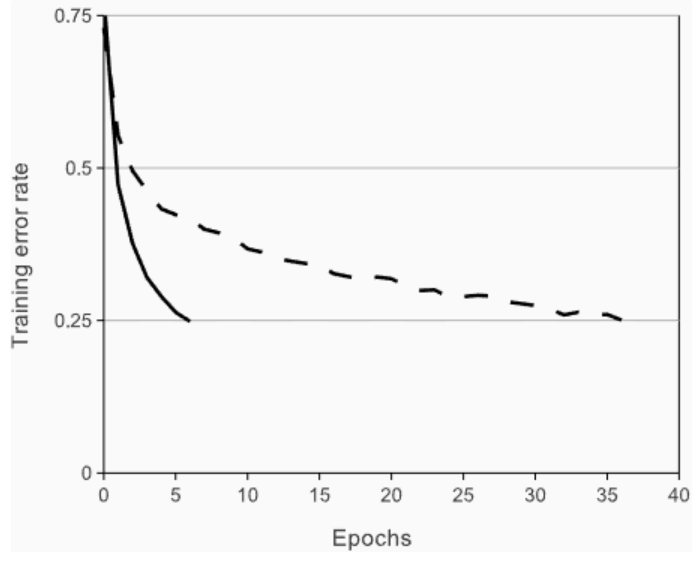


图 1：一个使用 ReLU 激活函数的四层卷积神经网络（实线）在 CIFAR-10 数据集上达到 25% 的训练错误率的速度，比使用 tanh 激活函数的同等网络（虚线）快六倍。每个网络的学习率都是独立选择的，以尽可能加快训练速度。没有使用任何正则化。此处展示的效果大小会随网络架构而变化，但使用 ReLU 激活函数的网络始终比使用饱和神经元的同等网络学习速度快数倍。

3.2 在多个 GPU 上进行训练

单个 GTX 580 GPU 只有 3GB 显存，这限制了其上可训练网络的最大规模。事实证明，120 万个训练样本足以训练出规模过大、无法在单个 GPU 上运行的网络。因此，我们将网络分布在两个 GPU 上。目前的 GPU 特别适合跨 GPU 并行化，因为它们可以直接读写彼此的内存，而无需经过主机内存。我们采用的并行化方案本质上是将一半的内核（或神经元）放在每个 GPU 上，并有一个额外的技巧：GPU 之间仅在某些层进行通信。这意味着，例如，第 3 层的内核从第 2 层的所有内核映射中获取输入。然而，第 4 层的内核仅从位于同一 GPU 上的第 3 层内核映射中获取输入。选择连接模式是交叉验证的一个难题，但这种方法使我们能够精确地调整通信量，直到它成为计算量中可接受的一部分。最终得到的架构与 Cire¸san 等人[5]使用的“列式”CNN 架构有些类似，不同之处在于我们的列并非相互独立（见图 2）。与每个卷积层使用一半卷积核并在单个 GPU 上训练的网络相比，该方案分别将 top-1 和 top-5 错误率降低了 1.7%和 1.2%。双 GPU 网络的训练时间略少于单 GPU 网络。

[2] 在最终卷积层中，单 GPU 网络实际上与双 GPU 网络拥有相同数量的卷积核。这是因为网络的大部分参数都位于第一个全连接层，而该全连接层以最后一个卷积层作为输入。因此，为了使两个网络拥有大致相同的参数数量，我们并没有将最终卷积层（以及后续的全连接层）的大小减半。所以，由于单 GPU 网络的大小大于双 GPU 网络的“一半”，因此这种比较结果对单 GPU 网络有利。

3.3 Local Response Normalization

ReLUs have the desirable property that they do not require input normalization to prevent them from saturating. If at least some training examples produce a positive input to a ReLU, learning will happen in that neuron. However, we still find that the following local normalization scheme aids generalization. Denoting by $a^i_{x,y}$ the activity of a neuron computed by applying kernel i at position (x, y) and then applying the ReLU nonlinearity, the response-normalized activity $b^i_{x,y}$ is given by the expression

$$b^i_{x,y} = a^i_{x,y} / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2 \right)^\beta$$

where the sum runs over n "adjacent" kernel maps at the same spatial position, and N is the total number of kernels in the layer. The ordering of the kernel maps is of course arbitrary and determined before training begins. This sort of response normalization implements a form of lateral inhibition inspired by the type found in real neurons, creating competition for big activities amongst neuron outputs computed using different kernels. The constants k, n, α, and β are hyper-parameters whose values are determined using a validation set; we used $k = 2$, $n = 5$, $\alpha = 10^{-4}$, and $\beta = 0.75$. We applied this normalization after applying the ReLU nonlinearity in certain layers (see Section 3.5).

This scheme bears some resemblance to the local contrast normalization scheme of Jarrett et al. [11], but ours would be more correctly termed "brightness normalization", since we do not subtract the mean activity. Response normalization reduces our top-1 and top-5 error rates by 1.4% and 1.2%, respectively. We also verified the effectiveness of this scheme on the CIFAR-10 dataset: a four-layer CNN achieved a 13% test error rate without normalization and 11% with normalization[3].

3.4 Overlapping Pooling

Pooling layers in CNNs summarize the outputs of neighboring groups of neurons in the same kernel map. Traditionally, the neighborhoods summarized by adjacent pooling units do not overlap (e.g., [17, 11, 4]). To be more precise, a pooling layer can be thought of as consisting of a grid of pooling units spaced s pixels apart, each summarizing a neighborhood of size $z \times z$ centered at the location of the pooling unit. If we set $s = z$, we obtain traditional local pooling as commonly employed in CNNs. If we set $s < z$, we obtain overlapping pooling. This is what we use throughout our network, with $s = 2$ and $z = 3$. This scheme reduces the top-1 and top-5 error rates by 0.4% and 0.3%, respectively, as compared with the non-overlapping scheme $s = 2, z = 2$, which produces output of equivalent dimensions. We generally observe during training that models with overlapping pooling find it slightly more difficult to overfit.

3.5 Overall Architecture

Now we are ready to describe the overall architecture of our CNN. As depicted in Figure 2, the net contains eight layers with weights; the first five are convolutional and the remaining three are fully-connected. The output of the last fully-connected layer is fed to a 1000-way softmax which produces a distribution over the 1000 class labels. Our network maximizes the multinomial logistic regression objective, which is equivalent to maximizing the average across training cases of the log-probability of the correct label under the prediction distribution.

The kernels of the second, fourth, and fifth convolutional layers are connected only to those kernel maps in the previous layer which reside on the same GPU (see Figure 2). The kernels of the third convolutional layer are connected to all kernel maps in the second layer. The neurons in the fully-connected layers are connected to all neurons in the previous layer. Response-normalization layers follow the first and second convolutional layers. Max-pooling layers, of the kind described in Section 3.4, follow both response-normalization layers as well as the fifth convolutional layer. The ReLU non-linearity is applied to the output of every convolutional and fully-connected layer.

The first convolutional layer filters the $224 \times 224 \times 3$ input image with 96 kernels of size $11 \times 11 \times 3$ with a stride of 4 pixels (this is the distance between the receptive field centers of neighboring

[3]We cannot describe this network in detail due to space constraints, but it is specified precisely by the code and parameter files provided here: http://code.google.com/p/cuda-convnet/.

3.3 局部响应归一化

ReLU 的一个优点是，它不需要输入归一化来防止饱和。只要至少有一些训练样本对 ReLU 产生正输入，该神经元就会发生学习。然而，我们仍然发现以下局部归一化方案有助于泛化。用 a 表示在位置 (x, y) 应用核 i 并应用 ReLU 非线性激活函数后计算得到的神经元活动，则响应归一化活动 bis 由以下表达式给出：

$$b_{x,y}^{i} = a_{x,y}^{i} \Big/ \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} \left(a_{x,y}^{j}\right)^{2}\right)^{\beta}$$

其中求和是对同一空间位置的 n 个“相邻”核映射进行的，N 是该层中核的总数。核映射的顺序当然是任意的，并且在训练开始前就已确定。这种响应归一化实现了一种受真实神经元中存在的侧抑制形式，它使得使用不同核计算出的神经元输出之间对高活动性的竞争。常数 k、n、α 和 β 是超参数，其值使用验证集确定；我们使用了 k = 2、n = 5、α = 10 和 β = 0.75。我们在某些层应用 ReLU 非线性激活函数之后应用了这种归一化（参见第 3.5 节）。

该方案与 Jarrett 等人 [11] 的局部对比度归一化方案有些相似，但更准确地说，我们的方案应该称为“亮度归一化”，因为我们没有减去平均活动值。响应归一化分别将我们的前 1 和前 5 错误率降低了 1.4% 和 1.2%。我们还在 CIFAR-10 数据集上验证了该方案的有效性：一个四层 CNN 在未进行归一化的情况下测试错误率为 13%，而进行归一化后则降至 11%。

3.4 重叠池

在卷积神经网络（CNN）中，池化层将相邻神经元组的输出汇总到同一个核映射中。传统上，相邻池化单元汇总的邻域互不重叠（例如，[17, 11, 4]）。更准确地说，池化层可以被视为由间隔 s 像素的池化单元网格组成，每个池化单元汇总一个以自身为中心、大小为 z × z 的邻域。如果令 s = z，则得到 CNN 中常用的传统局部池化。如果令 s < z，则得到重叠池化。我们在整个网络中都使用了重叠池化，其中 s = 2，z = 3。与输出维度相同的非重叠池化方案（s = 2，z = 2）相比，该方案分别将 top-1 和 top-5 错误率降低了 0.4% 和 0.3%。我们在训练过程中通常观察到，使用重叠池化的模型更不容易过拟合。

3.5 总体架构

现在我们可以描述我们卷积神经网络（CNN）的整体架构了。如图 2 所示，该网络包含八层带权重的层；前五层是卷积层，后三层是全连接层。最后一个全连接层的输出被送入一个 1000 路 softmax 函数，生成一个包含 1000 个类别标签的分布。我们的网络最大化多项式逻辑回归目标函数，这等价于最大化训练样本中预测分布下正确标签的对数概率的平均值。

第二、第四和第五卷积层的卷积核仅与位于同一 GPU 上的前一层卷积核映射相连（见图 2）。第三卷积层的卷积核与第二层的所有卷积核映射相连。全连接层中的神经元与前一层的所有神经元相连。第一、二卷积层之后是响应归一化层。响应归一化层以及第五卷积层之后是最大池化层（详见 3.4 节）。ReLU 非线性激活函数应用于每个卷积层和全连接层的输出。

第一个卷积层使用 96 个大小为 11 × 11 × 3 的卷积核对 224 × 224 × 3 的输入图像进行滤波，步长为 4 像素（这是相邻卷积核感受野中心之间的距离）。

---

[2] 由于篇幅限制，我们无法详细描述该网络，但它已通过此处提供的代码和参数文件进行了精确定义：http://code.google.com/p/cuda-convnet/。

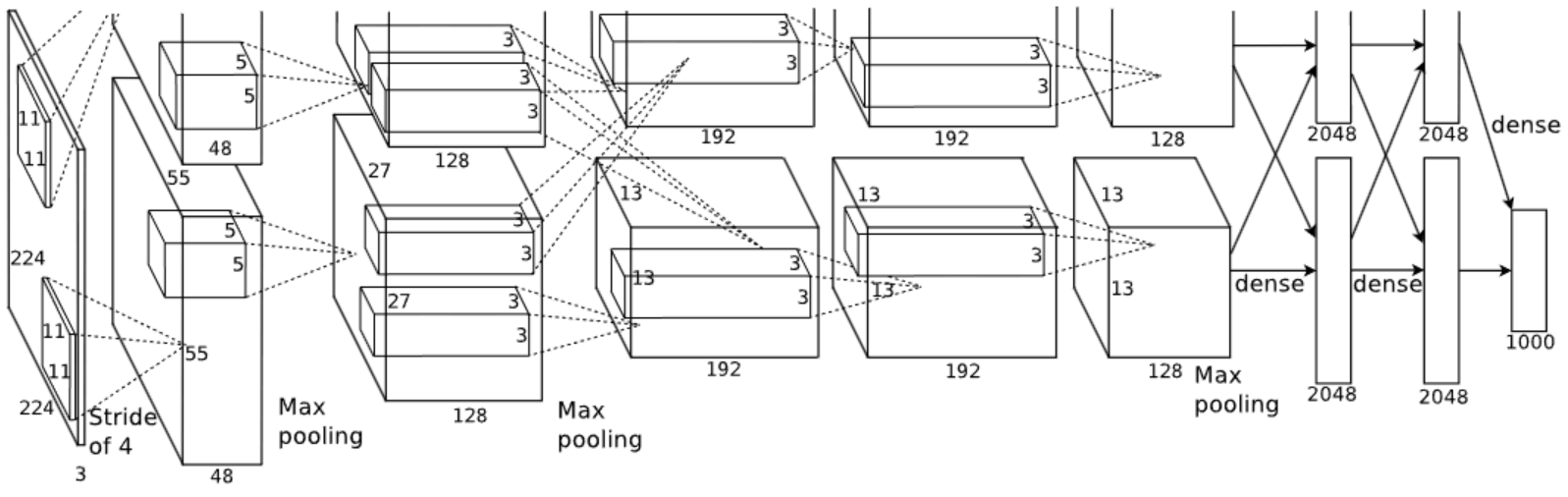


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

neurons in a kernel map). The second convolutional layer takes as input the (response-normalized and pooled) output of the first convolutional layer and filters it with 256 kernels of size $5 \times 5 \times 48$. The third, fourth, and fifth convolutional layers are connected to one another without any intervening pooling or normalization layers. The third convolutional layer has 384 kernels of size $3 \times 3 \times 256$ connected to the (normalized, pooled) outputs of the second convolutional layer. The fourth convolutional layer has 384 kernels of size $3 \times 3 \times 192$, and the fifth convolutional layer has 256 kernels of size $3 \times 3 \times 192$. The fully-connected layers have 4096 neurons each.

4 Reducing Overfitting

Our neural network architecture has 60 million parameters. Although the 1000 classes of ILSVRC make each training example impose 10 bits of constraint on the mapping from image to label, this turns out to be insufficient to learn so many parameters without considerable overfitting. Below, we describe the two primary ways in which we combat overfitting.

4.1 Data Augmentation

The easiest and most common method to reduce overfitting on image data is to artificially enlarge the dataset using label-preserving transformations (e.g., [25, 4, 5]). We employ two distinct forms of data augmentation, both of which allow transformed images to be produced from the original images with very little computation, so the transformed images do not need to be stored on disk. In our implementation, the transformed images are generated in Python code on the CPU while the GPU is training on the previous batch of images. So these data augmentation schemes are, in effect, computationally free.

The first form of data augmentation consists of generating image translations and horizontal reflections. We do this by extracting random 224×224 patches (and their horizontal reflections) from the 256×256 images and training our network on these extracted patches[4]. This increases the size of our training set by a factor of 2048, though the resulting training examples are, of course, highly interdependent. Without this scheme, our network suffers from substantial overfitting, which would have forced us to use much smaller networks. At test time, the network makes a prediction by extracting five 224×224 patches (the four corner patches and the center patch) as well as their horizontal reflections (hence ten patches in all), and averaging the predictions made by the network's softmax layer on the ten patches.

The second form of data augmentation consists of altering the intensities of the RGB channels in training images. Specifically, we perform PCA on the set of RGB pixel values throughout the ImageNet training set. To each training image, we add multiples of the found principal components,

[4]This is the reason why the input images in Figure 2 are $224 \times 224 \times 3$-dimensional.

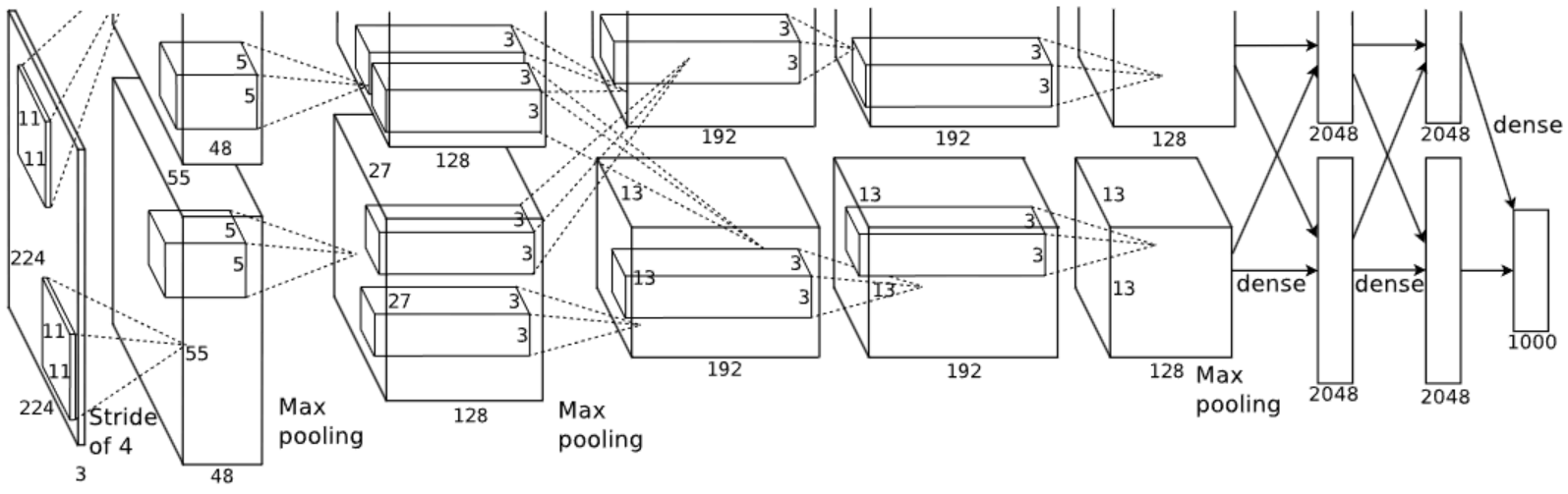


图 2：我们 CNN 的架构示意图，明确展示了两个 GPU 之间的职责划分。一个 GPU 运行图中上方的层，另一个 GPU 运行图中下方的层。两个 GPU 仅在特定层进行通信。网络的输入维度为 150,528，其余各层的神经元数量为 253,440–186,624–64,896–64,896–43,264–4096–4096–1000。

（内核映射中的神经元）。第二个卷积层以第一个卷积层（经过响应归一化和池化处理）的输出作为输入，并使用 256 个大小为 5 × 5 × 48 的卷积核进行滤波。第三、第四和第五个卷积层彼此连接，中间没有任何池化层或归一化层。第三个卷积层有 384 个大小为 3 × 3 × 256 的卷积核，连接到第二个卷积层（经过归一化和池化处理）的输出。第四个卷积层有 384 个大小为 3 × 3 × 192 的卷积核，第五个卷积层有 256 个大小为 3 × 3 × 192 的卷积核。每个全连接层有 4096 个神经元。

4　减少过拟合

我们的神经网络架构拥有 6000 万个参数。尽管 ILSVRC 的 1000 个类别使得每个训练样本对图像到标签的映射施加 10 位约束，但这仍然不足以在不出现严重过拟合的情况下学习如此多的参数。下面，我们将介绍我们对抗过拟合的两种主要方法。

4.1　数据增强

减少图像数据过拟合最简单且最常用的方法是使用保标签变换（例如[25, 4, 5]）人为地扩充数据集。我们采用了两种不同的数据增强方法，这两种方法都允许以极少的计算量从原始图像生成变换后的图像，因此变换后的图像无需存储在磁盘上。在我们的实现中，变换后的图像是在 CPU 上使用 Python 代码生成的，而 GPU 则在训练前一批图像。因此，这些数据增强方案实际上是零计算成本的。

第一种数据增强方法是生成图像平移和水平反射。我们通过从 256×256 的图像中随机提取 224×224 的图像块（及其水平反射），并使用这些提取的图像块训练我们的网络来实现这一点。这使得我们的训练集规模扩大了 2048 倍，但由此产生的训练样本之间当然存在高度依赖性。如果没有这种方法，我们的网络会遭受严重的过拟合，这将迫使我们使用规模小得多的网络。在测试阶段，网络通过提取五个 224×224 的图像块（四个角块和一个中心块）及其水平反射（总共十个图像块）来进行预测，并将网络 softmax 层对这十个图像块的预测结果取平均值。

第二种数据增强方法是改变训练图像中 RGB 通道的强度。具体来说，我们对 ImageNet 训练集中所有 RGB 像素值进行主成分分析（PCA）。对于每张训练图像，我们添加多个找到的主成分。

[4] 这就是为什么图 2 中的输入图像是 224 × 224 × 3 维的原因。

with magnitudes proportional to the corresponding eigenvalues times a random variable drawn from a Gaussian with mean zero and standard deviation 0.1. Therefore to each RGB image pixel $I_{xy} = [I^R_{xy}, I^G_{xy}, I^B_{xy}]^T$ we add the following quantity:

$$[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$$

where $\mathbf{p}_i$ and λ_i are ith eigenvector and eigenvalue of the 3×3 covariance matrix of RGB pixel values, respectively, and α_i is the aforementioned random variable. Each α_i is drawn only once for all the pixels of a particular training image until that image is used for training again, at which point it is re-drawn. This scheme approximately captures an important property of natural images, namely, that object identity is invariant to changes in the intensity and color of the illumination. This scheme reduces the top-1 error rate by over 1%.

4.2 Dropout

Combining the predictions of many different models is a very successful way to reduce test errors [1, 3], but it appears to be too expensive for big neural networks that already take several days to train. There is, however, a very efficient version of model combination that only costs about a factor of two during training. The recently-introduced technique, called "dropout" [10], consists of setting to zero the output of each hidden neuron with probability 0.5. The neurons which are "dropped out" in this way do not contribute to the forward pass and do not participate in back-propagation. So every time an input is presented, the neural network samples a different architecture, but all these architectures share weights. This technique reduces complex co-adaptations of neurons, since a neuron cannot rely on the presence of particular other neurons. It is, therefore, forced to learn more robust features that are useful in conjunction with many different random subsets of the other neurons. At test time, we use all the neurons but multiply their outputs by 0.5, which is a reasonable approximation to taking the geometric mean of the predictive distributions produced by the exponentially-many dropout networks.

We use dropout in the first two fully-connected layers of Figure 2. Without dropout, our network exhibits substantial overfitting. Dropout roughly doubles the number of iterations required to converge.

5 Details of learning

We trained our models using stochastic gradient descent with a batch size of 128 examples, momentum of 0.9, and weight decay of 0.0005. We found that this small amount of weight decay was important for the model to learn. In other words, weight decay here is not merely a regularizer: it reduces the model's training error. The update rule for weight w was

$$\begin{aligned} v_{i+1} &:= 0.9 \cdot v_i - 0.0005 \cdot \epsilon \cdot w_i - \epsilon \cdot \left\langle \frac{\partial L}{\partial w}\Big|_{w_i} \right\rangle_{D_i} \\ w_{i+1} &:= w_i + v_{i+1} \end{aligned}$$

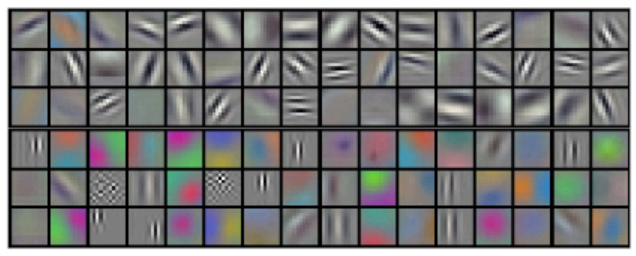


Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

where i is the iteration index, v is the momentum variable, ϵ is the learning rate, and $\left\langle \frac{\partial L}{\partial w}\big|_{w_i} \right\rangle_{D_i}$ is the average over the ith batch D_i of the derivative of the objective with respect to w, evaluated at w_i.

We initialized the weights in each layer from a zero-mean Gaussian distribution with standard deviation 0.01. We initialized the neuron biases in the second, fourth, and fifth convolutional layers, as well as in the fully-connected hidden layers, with the constant 1. This initialization accelerates the early stages of learning by providing the ReLUs with positive inputs. We initialized the neuron biases in the remaining layers with the constant 0.

We used an equal learning rate for all layers, which we adjusted manually throughout training. The heuristic which we followed was to divide the learning rate by 10 when the validation error rate stopped improving with the current learning rate. The learning rate was initialized at 0.01 and

其大小与相应的特征值成正比，并乘以一个从均值为零、标准差为 0.1 的高斯分布中抽取的随机变量。因此，对于每个 RGB 图像像素 I= [I, I, I]，我们添加以下量：

$$[p, p, p][\alpha\lambda, \alpha\lambda, \alpha\lambda]$$

其中，p 和 λ 分别是 RGB 像素值 3 × 3 协方差矩阵的第 i 个特征向量和特征值，α 是前述的随机变量。对于特定的训练图像，每个 α 值仅抽取一次，直到该图像再次用于训练时才会重新抽取。该方案近似地捕捉了自然图像的一个重要特性，即物体身份对光照强度和颜色的变化不敏感。该方案将 top-1 错误率降低了 1% 以上。

4.2 辍学

结合多个不同模型的预测结果是降低测试误差的有效方法[1, 3]，但对于训练本身就需要数天时间的大型神经网络来说，这种方法似乎成本过高。然而，有一种非常高效的模型组合方法，其训练成本仅为传统方法的一半左右。这种最近提出的技术称为“dropout”[10]，它以 0.5 的概率将每个隐藏神经元的输出置零。以这种方式“dropout”的神经元不会参与前向传播，也不会参与反向传播。因此，每次输入数据时，神经网络都会采样不同的网络架构，但所有这些架构共享权重。由于神经元不能依赖于特定其他神经元的存在，因此该技术减少了神经元之间复杂的协同适应。因此，神经元被迫学习更鲁棒的特征，这些特征可以与许多其他神经元的随机子集结合使用。在测试时，我们使用所有神经元，但将它们的输出乘以 0.5，这对于取指数级数量的 dropout 网络产生的预测分布的几何平均值来说是一个合理的近似值。

我们在图 2 的前两个全连接层中使用了 dropout。如果没有 dropout，我们的网络会出现严重的过拟合现象。Dropout 大约会使收敛所需的迭代次数增加一倍。

5 学习详情

我们使用随机梯度下降法训练模型，批大小为 128 个样本，动量为 0.9，权重衰减为 0.0005。我们发现，如此小的权重衰减对于模型的学习至关重要。换句话说，这里的权重衰减不仅仅是一个正则化项：它还能降低模型的训练误差。权重 w 的更新规则如下：

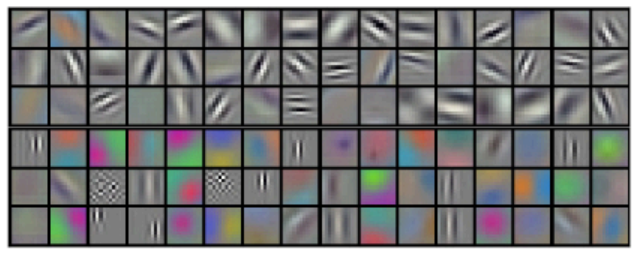


图 3：第一卷积层在 224×224×3 输入图像上学习到的 96 个大小为 11×11×3 的卷积核。前 48 个卷积核在 GPU 1 上学习，后 48 个卷积核在 GPU 2 上学习。详情请参见 6.1 节。

$$v := 0.9 \cdot v - 0.0005 \cdot \cdot w - \cdot \left\langle \frac{\partial L}{\partial w} \middle|_{w} \right\rangle_{D}$$

$$w := w + v$$

其中 i 是迭代索引，v 是动量变量，是学习率，$\left\langle \frac{\partial L}{\partial w} \middle|_{w} \right\rangle_{D}$ 是第 i 批次的平均值 Dof 目标函数对 w 的导数，在 w 处评估。

我们使用均值为零、标准差为 0.01 的高斯分布初始化每一层的权重。我们将第二、四、五卷积层以及全连接隐藏层的神经元偏置初始化为常数 1。这种初始化方式通过为 ReLU 函数提供正输入来加速学习的早期阶段。其余层的神经元偏置初始化为常数 0。

我们对所有层使用了相同的学习率，并在整个训练过程中手动调整学习率。我们遵循的启发式方法是：当验证错误率不再随当前学习率改善时，将学习率除以 10。学习率的初始值为 0.01。

reduced three times prior to termination. We trained the network for roughly 90 cycles through the training set of 1.2 million images, which took five to six days on two NVIDIA GTX 580 3GB GPUs.

6 Results

Our results on ILSVRC-2010 are summarized in Table 1. Our network achieves top-1 and top-5 test set error rates of **37.5%** and **17.0%**[5]. The best performance achieved during the ILSVRC-2010 competition was 47.1% and 28.2% with an approach that averages the predictions produced from six sparse-coding models trained on different features [2], and since then the best published results are 45.7% and 25.7% with an approach that averages the predictions of two classifiers trained on Fisher Vectors (FVs) computed from two types of densely-sampled features [24].

Model	**Top-1**	**Top-5**
Sparse coding [2]	*47.1%*	*28.2%*
SIFT + FVs [24]	*45.7%*	*25.7%*
CNN	**37.5%**	**17.0%**

Table 1: Comparison of results on ILSVRC-2010 test set. In *italics* are best results achieved by others.

We also entered our model in the ILSVRC-2012 competition and report our results in Table 2. Since the ILSVRC-2012 test set labels are not publicly available, we cannot report test error rates for all the models that we tried. In the remainder of this paragraph, we use validation and test error rates interchangeably because in our experience they do not differ by more than 0.1% (see Table 2). The CNN described in this paper achieves a top-5 error rate of 18.2%. Averaging the predictions of five similar CNNs gives an error rate of 16.4%. Training one CNN, with an extra sixth convolutional layer over the last pooling layer, to classify the entire ImageNet Fall 2011 release (15M images, 22K categories), and then "fine-tuning" it on ILSVRC-2012 gives an error rate of 16.6%. Averaging the predictions of two CNNs that were pre-trained on the entire Fall 2011 release with the aforementioned five CNNs gives an error rate of **15.3%**. The second-best contest entry achieved an error rate of 26.2% with an approach that averages the predictions of several classifiers trained on FVs computed from different types of densely-sampled features [7].

Model	**Top-1 (val)**	**Top-5 (val)**	**Top-5 (test)**
SIFT + FVs [7]	—	—	*26.2%*
1 CNN	40.7%	18.2%	—
5 CNNs	38.1%	16.4%	**16.4%**
1 CNN*	39.0%	16.6%	—
7 CNNs*	36.7%	15.4%	**15.3%**

Table 2: Comparison of error rates on ILSVRC-2012 validation and test sets. In *italics* are best results achieved by others. Models with an asterisk* were "pre-trained" to classify the entire ImageNet 2011 Fall release. See Section 6 for details.

Finally, we also report our error rates on the Fall 2009 version of ImageNet with 10,184 categories and 8.9 million images. On this dataset we follow the convention in the literature of using half of the images for training and half for testing. Since there is no established test set, our split necessarily differs from the splits used by previous authors, but this does not affect the results appreciably. Our top-1 and top-5 error rates on this dataset are **67.4%** and **40.9%**, attained by the net described above but with an additional, sixth convolutional layer over the last pooling layer. The best published results on this dataset are 78.1% and 60.9% [19].

6.1 Qualitative Evaluations

Figure 3 shows the convolutional kernels learned by the network's two data-connected layers. The network has learned a variety of frequency- and orientation-selective kernels, as well as various colored blobs. Notice the specialization exhibited by the two GPUs, a result of the restricted connectivity described in Section 3.5. The kernels on GPU 1 are largely color-agnostic, while the kernels on on GPU 2 are largely color-specific. This kind of specialization occurs during every run and is independent of any particular random weight initialization (modulo a renumbering of the GPUs).

[5]The error rates without averaging predictions over ten patches as described in Section 4.1 are 39.0% and 18.3%.

我们在训练过程中手动调整了参数，在终止前减少了三次。我们使用包含 120 万张图像的训练集对网络进行了大约 90 个循环的训练，这在两块 NVIDIA GTX 580 3GB GPU 上耗时五到六天。

6　结果

我们在 ILSVRC-2010 上的结果总结于表 1。我们的网络在测试集上实现了 37.5% 的 top-1 和 top-5 错误率，分别为 17.0%。ILSVRC-2010 竞赛期间的最佳性能为 47.1% 和 28.2%，该性能是通过对基于不同特征训练的六个稀疏编码模型的预测结果进行平均而实现的 [2]。此后，已发表的最佳结果为 45.7% 和 25.7%，该性能是通过对基于两种密集采样特征计算的 Fisher 向量 (FV) 训练的两个分类器的预测结果进行平均而实现的 [24]。

模型	前 1 名	前 5 名
稀疏编码 [2]	47.1%	28.2%
SIFT + FVs [24]	45.7%	25.7%
CNN	37.5%	17.0%

表 1：ILSVRC2010 测试集结果对比。斜体字为其他研究者取得的最佳结果。

我们还提交了模型参加 ILSVRC-2012 竞赛，并将结果列于表 2。由于 ILSVRC-2012 测试集的标签未公开，我们无法报告所有尝试模型的测试错误率。在本段的其余部分，我们将交替使用验证错误率和测试错误率，因为根据我们的经验，它们的差异不超过 0.1%（见表 2）。本文所述的 CNN 的 top-5 错误率为 18.2%。五个类似 CNN 的预测结果取平均值，得到错误率为 16.4%。训练一个 CNN，在最后一个池化层上增加一个额外的第六个卷积层，使其能够对整个 ImageNet 2011 年秋季版（1500 万张图像，2.2 万个类别）进行分类，然后在 ILSVRC-2012 数据集上进行微调，得到的错误率为 16.6%。将两个在 2011 年秋季发布版上预训练的 CNN 与上述五个 CNN 的预测结果取平均值，得到的错误率为 15.3%。第二好的参赛作品采用的方法，将几个分类器的预测结果取平均值，这些分类器是在从不同类型的密集采样特征计算出的 FV 上训练的 [7]，其错误率为 26.2%。

模型	前 1 名（验证集）	前 5 名（验证集）	前 5 名（测试集）
SIFT + FVs [7]			26.2%
1 CNN	40.7%	18.2%	—
5 个 CNN	38.1%	16.4%	16.4%
1 CNN*	39.0%	16.6%	—
7 个 CNN*	36.7%	15.4%	15.3%

表 2：ILSVRC-2012 验证集和测试集上的错误率比较。斜体字为其他研究者取得的最佳结果。标有星号*的模型经过“预训练”，能够对整个 ImageNet 2011 秋季版数据集进行分类。详情请参见第 6 节。

最后，我们还报告了在 2009 年秋季版 ImageNet 数据集上的错误率，该数据集包含 10,184 个类别和 890 万张图像。在这个数据集上，我们遵循文献中的惯例，将一半图像用于训练，一半用于测试。由于没有现成的测试集，我们的划分方式必然与先前作者使用的划分方式有所不同，但这不会对结果产生显著影响。我们在此数据集上的 top-1 和 top-5 错误率分别为 67.4% 和 40.9%，这是通过上述网络实现的，但需要在最后一个池化层上增加一个额外的第六个卷积层。该数据集上已发表的最佳结果分别为 78.1% 和 60.9% [19]。

6.1　定性评价

图 3 展示了网络两个数据连接层学习到的卷积核。该网络学习了多种频率选择性和方向选择性卷积核，以及各种颜色的色块。请注意两个 GPU 所展现出的特化特性，这是 3.5 节中描述的受限连接的结果。GPU 1 上的卷积核在很大程度上与颜色无关，而 GPU 2 上的卷积核则在很大程度上与颜色相关。这种特化特性在每次运行中都会出现，并且与任何特定的随机权重初始化无关（GPU 的重新编号除外）。

[5] 如果不按照第 4.1 节所述对十个块进行平均预测，则错误率分别为 39.0% 和 18.3%。

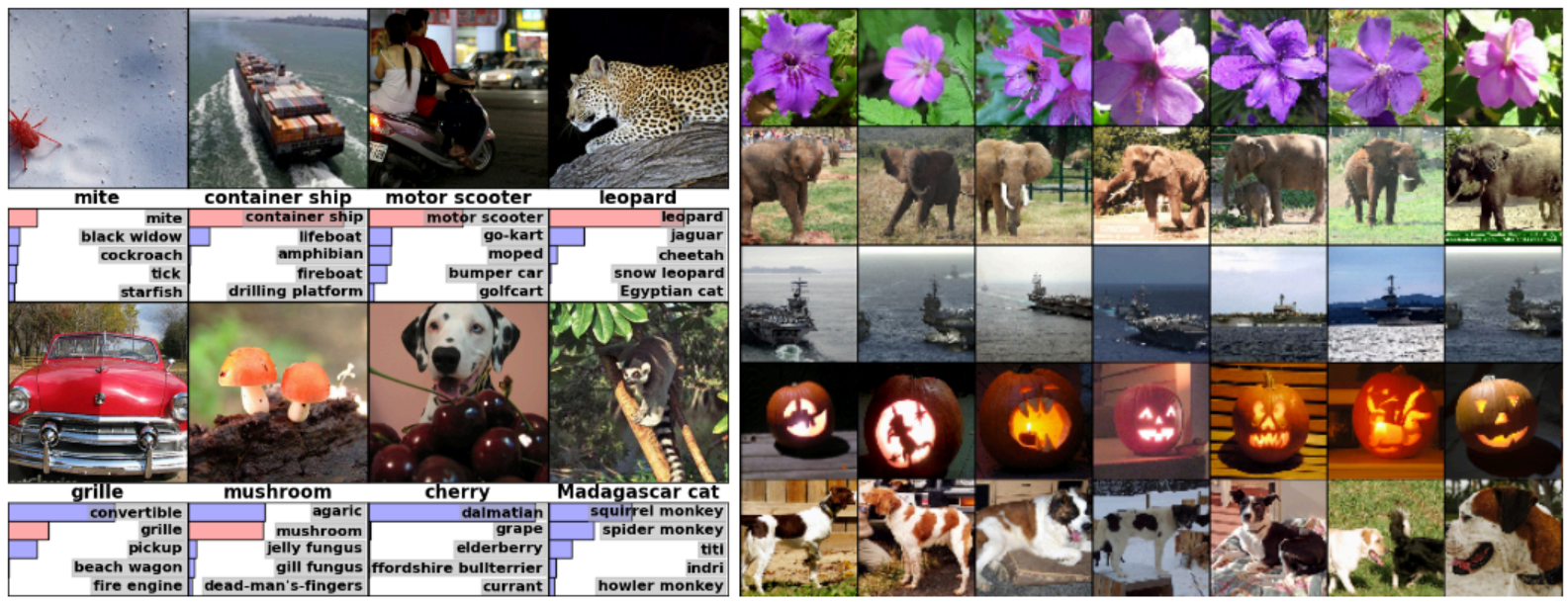


Figure 4: **(Left)** Eight ILSVRC-2010 test images and the five labels considered most probable by our model. The correct label is written under each image, and the probability assigned to the correct label is also shown with a red bar (if it happens to be in the top 5). **(Right)** Five ILSVRC-2010 test images in the first column. The remaining columns show the six training images that produce feature vectors in the last hidden layer with the smallest Euclidean distance from the feature vector for the test image.

In the left panel of Figure 4 we qualitatively assess what the network has learned by computing its top-5 predictions on eight test images. Notice that even off-center objects, such as the mite in the top-left, can be recognized by the net. Most of the top-5 labels appear reasonable. For example, only other types of cat are considered plausible labels for the leopard. In some cases (grille, cherry) there is genuine ambiguity about the intended focus of the photograph.

Another way to probe the network's visual knowledge is to consider the feature activations induced by an image at the last, 4096-dimensional hidden layer. If two images produce feature activation vectors with a small Euclidean separation, we can say that the higher levels of the neural network consider them to be similar. Figure 4 shows five images from the test set and the six images from the training set that are most similar to each of them according to this measure. Notice that at the pixel level, the retrieved training images are generally not close in L2 to the query images in the first column. For example, the retrieved dogs and elephants appear in a variety of poses. We present the results for many more test images in the supplementary material.

Computing similarity by using Euclidean distance between two 4096-dimensional, real-valued vectors is inefficient, but it could be made efficient by training an auto-encoder to compress these vectors to short binary codes. This should produce a much better image retrieval method than applying auto-encoders to the raw pixels [14], which does not make use of image labels and hence has a tendency to retrieve images with similar patterns of edges, whether or not they are semantically similar.

7 Discussion

Our results show that a large, deep convolutional neural network is capable of achieving record-breaking results on a highly challenging dataset using purely supervised learning. It is notable that our network's performance degrades if a single convolutional layer is removed. For example, removing any of the middle layers results in a loss of about 2% for the top-1 performance of the network. So the depth really is important for achieving our results.

To simplify our experiments, we did not use any unsupervised pre-training even though we expect that it will help, especially if we obtain enough computational power to significantly increase the size of the network without obtaining a corresponding increase in the amount of labeled data. Thus far, our results have improved as we have made our network larger and trained it longer but we still have many orders of magnitude to go in order to match the infero-temporal pathway of the human visual system. Ultimately we would like to use very large and deep convolutional nets on video sequences where the temporal structure provides very helpful information that is missing or far less obvious in static images.

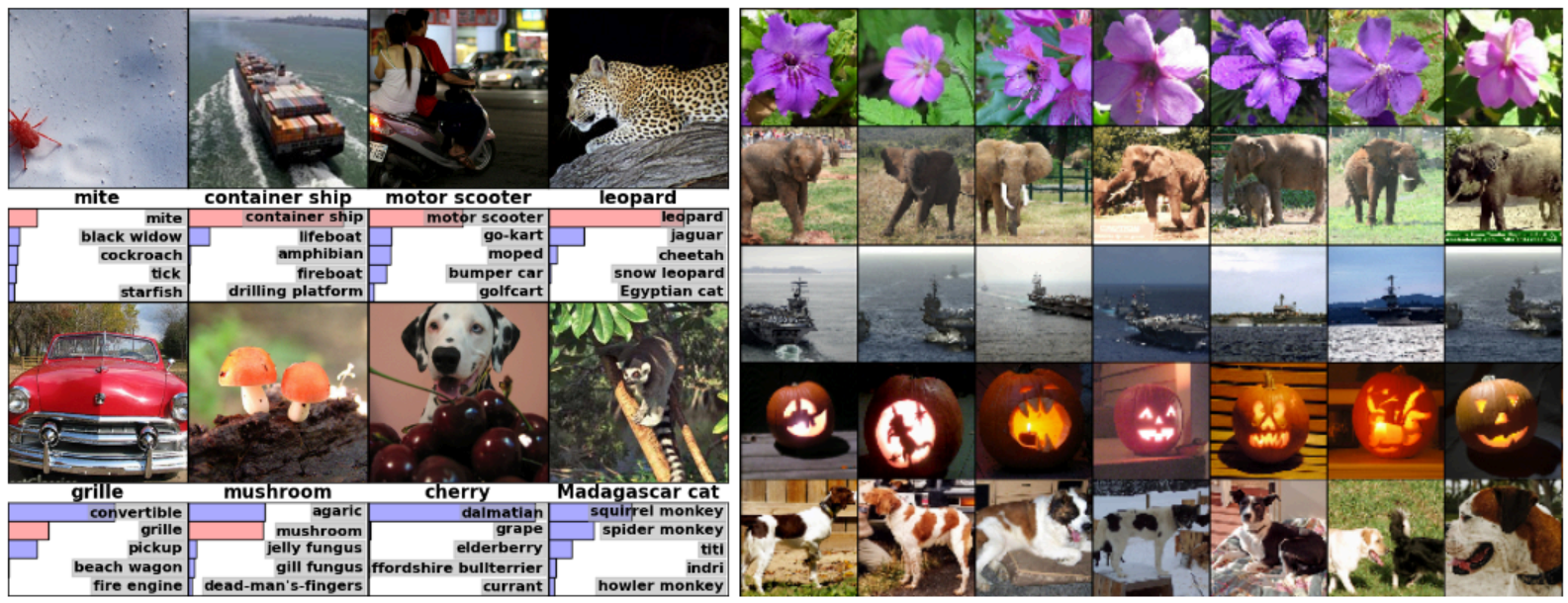


图 4：（左）八张 ILSVRC-2010 测试图像以及我们模型认为最有可能的五个标签。每张图像下方都标注了正确的标签，并且用红色条形图显示了分配给正确标签的概率（如果它恰好在前五名之列）。（右）第一列为五张 ILSVRC-2010 测试图像。其余列显示了六张训练图像，这些图像在最后一个隐藏层中生成的特征向量与测试图像的特征向量之间的欧氏距离最小。

在图 4 的左侧面板中，我们通过计算网络在八张测试图像上的前 5 个预测结果，对网络的学习成果进行定性评估。请注意，即使是偏离中心的物体，例如左上角的螨虫，也能被网络识别出来。大多数前 5 个标签看起来都很合理。例如，只有其他类型的猫才被认为是豹子的合理标签。在某些情况下（例如烤架、樱桃），照片的预期焦点确实存在歧义。

另一种探究网络视觉知识的方法是考察图像在最后一个（4096 维）隐藏层中引起的特征激活。如果两幅图像的特征激活向量欧氏距离较小，我们可以说神经网络的高层认为它们相似。图 4 展示了测试集中的五幅图像，以及根据此度量方法与每幅图像最相似的训练集中的六幅图像。值得注意的是，在像素级别上，检索到的训练图像与第一列中的查询图像在 L2 层上的相似度通常不高。例如，检索到的狗和大象呈现出各种不同的姿势。我们在补充材料中展示了更多测试图像的结果。

使用两个 4096 维实值向量之间的欧氏距离来计算相似度效率低下，但可以通过训练自编码器将这些向量压缩成短二进制代码来提高效率。这应该会产生比直接将自编码器应用于原始像素 [14] 更好的图像检索方法，因为后者没有利用图像标签，因此倾向于检索具有相似边缘模式的图像，而不管它们在语义上是否相似。

7　讨论

我们的结果表明，一个大型的深度卷积神经网络仅使用监督学习就能在极具挑战性的数据集上取得突破性成果。值得注意的是，如果移除单个卷积层，我们网络的性能就会下降。例如，移除任何中间层都会导致网络的 top-1 性能下降约 2%。因此，网络深度对于取得我们的成果至关重要。

为了简化实验，我们没有使用任何无监督预训练，尽管我们预期它会有所帮助，尤其是在我们获得足够的计算能力，可以在不相应增加标注数据量的情况下显著扩大网络规模的情况下。到目前为止，随着网络规模的扩大和训练时间的延长，我们的结果有所改进，但要达到人类视觉系统的颞下通路，我们还有很长的路要走。最终，我们希望在视频序列上使用非常大且深度的卷积神经网络，因为视频序列的时间结构提供了静态图像中缺失或不太明显的非常有用的信息。

References

[1] R.M. Bell and Y. Koren. Lessons from the netflix prize challenge. *ACM SIGKDD Explorations Newsletter*, 9(2):75–79, 2007.

[2] A. Berg, J. Deng, and L. Fei-Fei. Large scale visual recognition challenge 2010. www.image-net.org/challenges. 2010.

[3] L. Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[4] D. Cireşan, U. Meier, and J. Schmidhuber. Multi-column deep neural networks for image classification. *Arxiv preprint arXiv:1202.2745*, 2012.

[5] D.C. Cireşan, U. Meier, J. Masci, L.M. Gambardella, and J. Schmidhuber. High-performance neural networks for visual object classification. *Arxiv preprint arXiv:1102.0183*, 2011.

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.

[7] J. Deng, A. Berg, S. Satheesh, H. Su, A. Khosla, and L. Fei-Fei. *ILSVRC-2012*, 2012. URL `http://www.image-net.org/challenges/LSVRC/2012/`.

[8] L. Fei-Fei, R. Fergus, and P. Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. *Computer Vision and Image Understanding*, 106(1):59–70, 2007.

[9] G. Griffin, A. Holub, and P. Perona. Caltech-256 object category dataset. Technical Report 7694, California Institute of Technology, 2007. URL `http://authors.library.caltech.edu/7694`.

[10] G.E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R.R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.

[11] K. Jarrett, K. Kavukcuoglu, M. A. Ranzato, and Y. LeCun. What is the best multi-stage architecture for object recognition? In *International Conference on Computer Vision*, pages 2146–2153. IEEE, 2009.

[12] A. Krizhevsky. Learning multiple layers of features from tiny images. Master's thesis, Department of Computer Science, University of Toronto, 2009.

[13] A. Krizhevsky. Convolutional deep belief networks on cifar-10. *Unpublished manuscript*, 2010.

[14] A. Krizhevsky and G.E. Hinton. Using very deep autoencoders for content-based image retrieval. In *ESANN*, 2011.

[15] Y. Le Cun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, et al. Handwritten digit recognition with a back-propagation network. In *Advances in neural information processing systems*, 1990.

[16] Y. LeCun, F.J. Huang, and L. Bottou. Learning methods for generic object recognition with invariance to pose and lighting. In *Computer Vision and Pattern Recognition, 2004. CVPR 2004. Proceedings of the 2004 IEEE Computer Society Conference on*, volume 2, pages II–97. IEEE, 2004.

[17] Y. LeCun, K. Kavukcuoglu, and C. Farabet. Convolutional networks and applications in vision. In *Circuits and Systems (ISCAS), Proceedings of 2010 IEEE International Symposium on*, pages 253–256. IEEE, 2010.

[18] H. Lee, R. Grosse, R. Ranganath, and A.Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 609–616. ACM, 2009.

[19] T. Mensink, J. Verbeek, F. Perronnin, and G. Csurka. Metric Learning for Large Scale Image Classification: Generalizing to New Classes at Near-Zero Cost. In *ECCV - European Conference on Computer Vision*, Florence, Italy, October 2012.

[20] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proc. 27th International Conference on Machine Learning*, 2010.

[21] N. Pinto, D.D. Cox, and J.J. DiCarlo. Why is real-world visual object recognition hard? *PLoS computational biology*, 4(1):e27, 2008.

[22] N. Pinto, D. Doukhan, J.J. DiCarlo, and D.D. Cox. A high-throughput screening approach to discovering good forms of biologically inspired visual representation. *PLoS computational biology*, 5(11):e1000579, 2009.

[23] B.C. Russell, A. Torralba, K.P. Murphy, and W.T. Freeman. Labelme: a database and web-based tool for image annotation. *International journal of computer vision*, 77(1):157–173, 2008.

[24] J. Sánchez and F. Perronnin. High-dimensional signature compression for large-scale image classification. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1665–1672. IEEE, 2011.

[25] P.Y. Simard, D. Steinkraus, and J.C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, volume 2, pages 958–962, 2003.

[26] S.C. Turaga, J.F. Murray, V. Jain, F. Roth, M. Helmstaedter, K. Briggman, W. Denk, and H.S. Seung. Convolutional networks can learn to generate affinity graphs for image segmentation. *Neural Computation*, 22(2):511–538, 2010.

参考

[1] RM Bell 和 Y. Koren. 从 Netflix 大奖挑战赛中汲取的经验教训。ACM SIGKDD Explorations Newsletter，9(2):75–79，2007。[2] A. Berg、J. Deng 和 L. Fei-Fei. 2010 年大规模视觉识别挑战赛。www.imagenet.org/challenges。2010。

[3] L. Breiman. 随机森林. 机器学习, 45(1):5–32, 2001.

[4] D. Cire¸san、U. Meier 和 J. Schmidhuber。用于图像分类的多列深度神经网络。ArXiv 预印本 arXiv:1202.2745, 2012。

[5] DC Cire¸san、U. Meier、J. Masci、LM Gambardella 和 J. Schmidhuber。用于视觉对象分类的高性能神经网络。 Arxiv 预印本 arXiv:1102.0183, 2011。 [6] J. Deng、W. Dong、R. Socher、L.-J。 Li、K. Li 和 L. Fei-Fei。 ImageNet：大规模分层图像数据库。 CVPR09，2009。 [7] J. Deng、A. Berg、S. Satheesh、H. Su、A. Khosla 和 L. Fei-Fei。 ILSVRC-2012，2012。URL http://www.image-net.org/challenges/LSVRC/2012/。 [8] L. Fei-Fei、R. Fergus 和 P. Perona。从少量训练样本中学习生成式视觉模型：一种在 101 个对象类别上测试的增量贝叶斯方法。《计算机视觉与图像理解》，106(1):59–70，2007。[9] G. Griffin、A. Holub 和 P. Perona。《Caltech-256 对象类别数据集》。技术报告 7694，加州理工学院，2007。网址：http://authors.library.caltech.edu/7694。

[10] GE Hinton、N. Srivastava、A. Krizhevsky、I. Sutskever 和 RR Salakhutdinov。“通过防止特征检测器的协同适应来改进神经网络”。arXiv 预印本 arXiv:1207.0580，2012 年。[11] K. Jarrett、K. Kavukcuoglu、MA Ranzato 和 Y. LeCun。“什么是用于目标识别的最佳多阶段架构？”国际计算机视觉会议论文集，第 2146–2153 页。IEEE，2009 年。[12] A. Krizhevsky。“从微小图像中学习多层特征”。多伦多大学计算机科学系硕士论文，2009 年。

[13] A. Krizhevsky. 基于 CIFAR-10 的卷积深度信念网络。未发表的手稿，2010 年。[14] A. Krizhevsky 和 GE Hinton. 使用非常深的自编码器进行基于内容的图像检索。ESANN，2011 年。[15] Y. Le Cun、B. Boser、JS Denker、D. Henderson、RE Howard、W. Hubbard、LD Jackel 等。Hand-

利用反向传播网络进行书写数字识别。载于《神经信息处理系统进展》，1990 年。

[16] Y. LeCun、FJ Huang 和 L. Bottou. 具有不变性的通用对象识别学习方法姿态和光照。计算机视觉与模式识别，2004 年。CVPR 2004。2004 年 IEEE 计算机学会会议论文集，第 2 卷，第 II-97 页。IEEE，2004 年。

[17] Y. LeCun、K. Kavukcuoglu 和 C. Farabet。卷积网络及其在视觉中的应用。载于 2010 年 IEEE 国际电路与系统研讨会 (ISCAS) 会议录，第 253-256 页。IEEE，2010 年。

[18] H. Lee、R. Grosse、R. Ranganath 和 AY Ng。用于可扩展非超卷积神经网络的卷积深度信念网络可视化学习分层表示。载于第 26 届国际机器学习年会论文集，第 609-616 页。ACM，2009 年。

[19] T. Mensink、J. Verbeek、F. Perronnin 和 G. Csurka。大规模图像分类的度量学习cation：以近乎零成本泛化到新类别。发表于 ECCV - 欧洲计算机视觉会议，意大利佛罗伦萨，2012 年 10 月。

[20] V. Nair 和 GE Hinton. 修正线性单元改进受限玻尔兹曼机。载于第 27 届国际机器学习会议论文集，2010 年。

[21] N. Pinto、DD Cox 和 JJ DiCarlo. 为什么现实世界的视觉对象识别如此困难？PLoS 计算生物学，4(1):e27，2008 年。[22] N. Pinto、D. Doukhan、JJ DiCarlo 和 DD Cox. 一种用于发现高通量筛选方法的……

生物启发式视觉表征的良好形式。PLoS 计算生物学，5(11):e1000579，2009。

[23] BC Russell、A. Torralba、KP Murphy 和 WT Freeman。Labelme：用于图像标注的数据库和基于 Web 的工具。国际计算机视觉杂志，77(1):157–173，2008。

[24] J. Sánchez 和 F. Perronnin. 用于大规模图像分类的高维签名压缩。在 2011 年 IEEE 计算机视觉与模式识别会议 (CVPR) 上，第 1665-1672 页。IEEE，2011 年。

[25] PY Simard、D. Steinkraus 和 JC Platt。“卷积神经网络的最佳实践应用于……”视觉文档分析。第七届国际文档分析与识别会议论文集，第 2 卷，第 958-962 页，2003 年。

[26] SC Turaga、JF Murray、V. Jain、F. Roth、M. Helmstaedter、K. Briggman、W. Denk 和 HS Seung。Con-卷积神经网络可以学习生成用于图像分割的亲和图。神经计算，22(2):511–538, 2010。